UNREVIEWED WORKING PAPER / SYNTHETIC SIMULATION PILOT

Risk-Aware Adaptive Verification for Tool-Using Agents: A Controlled Simulation of Silent Errors and Ambiguous State Changes

Author: Roshaan Ahmed Khan | BSAI, Sindh Madressatul Islam University, Pakistan

Date: 10 November 2026 | Public working paper: v1.0 | Supplementary reproducibility archive

PUBLICATION STATUS: Preliminary working paper, not peer-reviewed. This study evaluates scripted policies, not LLM agents. Recent 2025–2026 related work has not been exhaustively verified.

Abstract

Tool-using AI agents may encounter explicit tool errors, plausible but incorrect responses, and ambiguous acknowledgments after state-changing operations. A verification policy can mitigate these risks, but checking every result increases tool traffic. We evaluate a predefined risk-aware verification controller in a controlled simulation of inventory workflows. Each of 1,600 deterministic synthetic scenarios involves two reads, a non-idempotent stock update, and an outcome report. We compare four scripted policies: no checks, bounded retry, verification after every operation, and selective verification based on observable anomalies and action risk. The primary outcome requires both correct final system state and a correct report. Selective verification achieved 85.12% task success versus 62.12% for no checks, a paired gain of +23.00 percentage points (95% CI +21.00 to +25.00). It used 4.11 total tool calls per scenario versus 6.13 for verify-all, but its success rate remained 14.88% lower. Naive retries produced 67 duplicate state changes after lost acknowledgments; selective verification produced none under the simulated conditions. These results demonstrate a controlled reliability-cost trade-off, not superior performance to full verification. Our fixed-rule agent, synthetic failure distribution, and idealized audit service limit external validity; LLM-backed, multi-domain tests are required before wider claims.

Keywords: Agent reliability; tool execution; fault injection; selective verification; safe retries; simulation; reproducibility

1. Introduction

Language-model-based agents extend text generation by invoking tools and modifying external environments. Tool calls enable more capable workflows but introduce execution failures that are not addressed by language fluency alone. In a multi-step task, a malformed response may be obvious, whereas stale data or an unacknowledged successful write may be difficult to diagnose. A simple retry may help when a call failed before execution yet cause a duplicate mutation when the original call actually succeeded.

Agent architectures have been studied through reasoning-and-action loops [1] and tool learning [2]. Feedback-based methods such as Reflexion [3] and externally supported critique [4] provide related foundations for correcting errors. Interactive evaluations including AgentBench [5], tau-bench [6], WebArena [7], and SWE-bench [8] demonstrate the importance of assessing agent behavior by task outcomes rather than answer plausibility alone.

The present paper reports a deliberately limited first experiment: a controlled simulation of scripted agent policies with explicit and silent tool faults. Its aim is to measure the reliability-cost trade-off of a

transparent verification rule under matched conditions and to validate the experimental apparatus for later LLM experiments. The word “agent” describes a tool-using workflow controller here; no language-model inference is performed.

We make three bounded contributions: (1) an auditable, deterministic simulator featuring silent data corruption and ambiguous commits; (2) four comparison policies implemented under identical failure scenarios; and (3) reproducible outcome measurements and paired bootstrap intervals. This work does not claim that adaptive verification is newly invented or that its benefits have been established for deployed autonomous systems.

2. Related Work and Novelty Status

2.1 Tool use, correction, and evaluation

ReAct [1] integrates language reasoning with interaction, providing a foundation for action-observation loops. Toolformer [2] focuses on language models learning tool usage. Reflexion [3] uses feedback to guide iterative behavior, while CRITIC [4] evaluates external-tool-assisted critique and revision. These approaches demonstrate that tool reasoning and feedback-based correction are established ideas. Accordingly, claiming the invention of self-verification would be unjustified.

AgentBench [5], tau-bench [6], WebArena [7], and SWE-bench [8] provide contrasting testbeds for interactive or outcome-grounded evaluation. Our pilot does not attempt to reproduce those benchmarks. Instead, it isolates tool-fault timing and independently known state transitions, which makes causal debugging straightforward but at the cost of lower realism.

2.2 Candidate research gap, not a confirmed novelty claim

A relevant next question is whether a risk policy that separates read anomalies from ambiguous non-idempotent writes can maintain reliability while avoiding unnecessary checks. This combination of fault categories, state verification, and verification overhead is our candidate angle. The full 2025–2026 literature and overlapping work on adaptive verification, operation idempotency, agent observability, and recovery must be examined before a novelty claim or conference submission. This manuscript is complete as a pilot study but not yet a fully contextualized state-of-the-art paper.

Prior research	Established contribution	Relationship to our pilot
ReAct [1]; Toolformer [2]	Tool use and action generation	Background; our controller is scripted
Reflexion [3]; CRITIC [4]	Feedback and self-correction	Related to verification/recovery
AgentBench [5]; tau-bench [6]	Agent interaction evaluation	Supports outcome-based evaluation
WebArena [7]; SWE-bench [8]	Realistic interactive tasks	Highlights realism gap in simulation
This pilot	Seeded tool failures and selective checking	Controlled empirical comparison; novelty pending

3. Research Questions and Hypotheses

RQ1. How much does selective verification improve end-to-end correctness over a no-check policy under the simulated fault distribution?

RQ2. How many tool calls does selective verification save relative to verifying every tool result, and what task-success penalty accompanies those savings?

RQ3. Can checking operation status before retry prevent duplicate changes when a write succeeds but its acknowledgment is lost?

H1 predicts higher success for selective verification than for no checks. H2 hypothesizes a favorable trade-off compared with full verification, without assuming equivalence. H3 predicts fewer duplicate mutations for the state-aware approach than naive retry. All hypotheses are interpreted only within the tested synthetic conditions.

4. Simulator and Verification Architecture

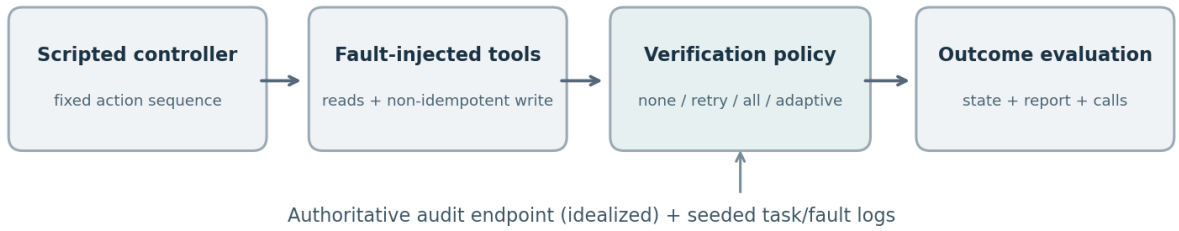


Figure 1. Experimental architecture. The decision-maker is scripted; authoritative audit endpoints are separate from the injected-fault tool responses.

4.1 Task model

Each scenario begins with an inventory stock quantity and item price. A scripted controller performs a stock read, a price read, and a stock decrement of one to five units. It then reports the expected post-update stock and the item price. Ground truth is held separately in the simulated tool environment. Task success requires both (a) exactly one intended state change and (b) a correct report based on the item's actual pre-operation state and price.

For this initial instrument, the task itself is fixed and the model does not decide which tool to invoke. This design isolates verification behavior from planning errors but does not measure genuine LLM agent competence.

4.2 Fault model

Operation	Fault class	Effect and observable response
Read	Missing result	Required value absent from a status-ok response
Read	Invalid result	Impossible negative value returned
Read	Silent corruption	Plausible numerical value differs from truth
Write	Explicit error	No mutation; error status returned
Write	Timeout before commit	No mutation; timeout reported
Write	Timeout after commit	Mutation applied; timeout reported
Write	Silent dropped commit	No mutation; success falsely reported

Fault probabilities are artificial, selected in advance for a stress-test pilot rather than estimated from production telemetry. Each read has probabilities 0.85 clean, 0.07 plausible corruption, 0.05 missing value, and 0.03 invalid value. The write has probability 0.80 clean and 0.05 each for explicit error, pre-commit timeout, post-commit timeout, and silent drop. These choices ensure a substantial minority of test scenarios contain observable or silent faults.

4.3 Four policies

Policy	Read behavior	Write behavior
B0 / no-check	Accept first response	Accept first write response
B1 / retry	Repeat invalid reads once	Repeat write on explicit error or timeout
B2 / verify-all	Audit both reads	Audit operation log; recover if not committed
B3 / adaptive	Audit absent/invalid reads only	Audit when high risk or non-success status; recover if needed

The adaptive rule treats a write as high risk when the absolute stock change is at least three units or the observed price is at least 150. This threshold is a predefined heuristic, not a learned risk score; it is fixed rather than tuned to test outcomes. On a timeout, adaptive recovery consults an operation-status audit endpoint before issuing any new write. A second attempt is made only when no first commit is recorded.

The audit services report authoritative state or whether an operation ID committed. They are assumed accurate and immediately consistent in this pilot. That assumption creates an intentionally optimistic reference for verify-all and must be relaxed in follow-up experiments.

5. Experimental Methods

5.1 Reproducibility and paired design

We generated 1,600 synthetic scenarios with Python's standard pseudorandom generator using seed 20261110. Scenario parameters and injected fault draws are identical for all four policies. Outcomes are evaluated against independent simulator state, not judged by a language model. The experiment exports a scenario JSONL file and one outcome row per scenario-policy pair. The code and exact fault probabilities accompany this manuscript.

Inventory starts between 15 and 100 items, item price ranges from 25 to 250 units, and each task decrements between one and five items. Each policy is run in its own isolated simulated tool environment, preventing contamination from other policies. Tool-call counts include original calls, retries, and audit checks. No monetary API cost, model tokens, or meaningful production latency is measured.

5.2 Outcome definitions

Primary outcome: task success is 1 only when the final stock is correct and the generated report matches ground truth; otherwise it is 0. Secondary outcomes are state correctness, report correctness, number of duplicate writes, mean calls, and mean additional verification calls. A duplicate write is identified when the inventory is decremented more than once for a single intended operation.

5.3 Statistical approach

For paired differences, each scenario contributes the difference between two policy outcomes. We resampled scenarios with replacement 3,000 times and used percentile 95% bootstrap intervals. These intervals describe uncertainty conditional on the chosen artificial scenario distribution. They do not estimate performance for all real-world agents or tools. We did not perform a formal non-inferiority test or preregister a non-inferiority margin; the paper therefore does not claim equivalence of selective and full verification.

6. Results

Policy	Task success	State correct	Report correct	Mean calls	Audit checks	Duplicates
no_check	62.12%	86.94%	70.88%	3.00	0.00	0
retry	79.00%	91.75%	85.69%	3.30	0.00	67
verify_all	100.00%	100.00%	100.00%	6.13	3.00	0
adaptive	85.12%	99.19%	85.69%	4.11	0.99	0

Table 2. Observed empirical results from the bundled scripted simulator. The 100% verify-all figure is a consequence of the reliable audit assumptions, not evidence of perfect agent reliability in practice.

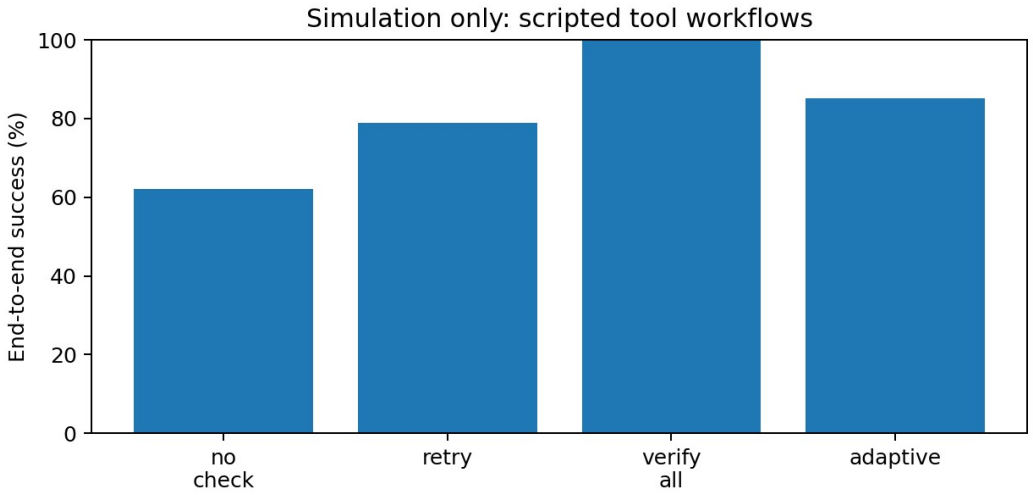


Figure 2. End-to-end task success for four policies, conditional on the synthetic test distribution.

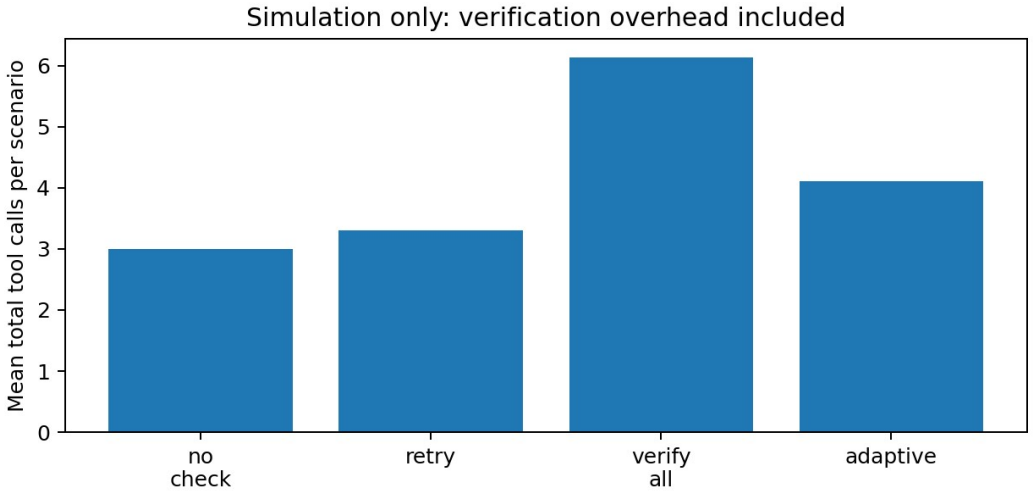


Figure 3. Mean total tool calls, including verification and retries. No inference-cost estimate is implied.

6.1 Selective verification versus no verification

Selective verification achieved 85.12% success compared with 62.12% for no checks. The paired difference was +23.00 percentage points (95% CI +21.00 to +25.00). This supports H1 within the simulated workflow and fault distribution. The controller also raised final-state correctness to 99.19%, although report correctness reached only 85.69% because plausible read corruption could remain unnoticed.

6.2 Verification overhead and accuracy trade-off

The adaptive policy used 4.11 calls per scenario against 6.13 for verify-all, a reduction of 32.96% in this simulated call-count proxy. The paired call difference was -2.021 calls (95% CI -2.047 to -1.992). However, adaptive task success was -14.88 percentage points (95% CI -16.62 to -13.19) relative to verify-all. Therefore H2 is supported only as a measurable resource-saving trade-off, not as reliability equivalence. Our current rule sacrifices too much accuracy to claim a universally superior policy.

6.3 Ambiguous commit safety

The naive retry policy produced 67 duplicate write events in 1,600 tasks. Each occurred in a simulated post-commit timeout scenario in which a second non-idempotent decrement was issued. Adaptive verification produced 0 duplicates. The paired duplicate-rate difference (retry minus adaptive) was +4.19 percentage points (95% CI +3.25 to +5.19). This supports H3 under the idealized operation-status readback assumption. The result illustrates why timeout handling needs different logic for operations that may already have committed.

6.4 Failure-mode interpretation

The held-out scenario generation produced 113 plausible stock-read corruptions and 123 plausible price-read corruptions. The adaptive policy audits schema-invalid reads but does not audit plausible values unless other conditions trigger a check. This limitation explains much of the remaining report error. Conversely, the high state-correctness rate indicates that operation-status checks are effective in the simplified simulated write environment. These observations describe the code's tested behavior; they should not be generalized to independent LLM agents.

7. Discussion

The results make the verification trade-off concrete. Compared with blindly trusting tools, selective checks improve task success. Compared with a reliable audit after every operation, they save calls but leave silent read corruption unresolved. This is a scientifically useful negative result: anomaly checks based on schema validity are insufficient when incorrect values still look plausible. A future policy should consider independent corroboration, state criticality, and explicit costs of incorrect decisions, while controlling false alerts and verification load.

Ambiguous write acknowledgments show a second issue. Retrying a failed-looking call is not necessarily safe when the side effect may have occurred. Operation identifiers, read-after-write confirmation, and idempotency semantics are practical mechanisms to examine further. Their role here is partly an implementation assumption rather than an original invention. Future comparisons should include idempotent APIs and at-least-once delivery semantics.

8. Threats to Validity and Research Ethics

External validity. The decision-maker is scripted, not a language model, and the task is a single synthetic inventory domain. There are no browser interactions, multi-agent coordination, or production tools.

Construct validity. Authoritative audit endpoints are idealized and immediately consistent. Verify-all succeeds perfectly by design when the audit is reliable, which overstates what a real-world verifier could guarantee.

Fault assumptions. Fault probabilities are synthetic and not empirically estimated from deployment logs. Results are sensitive to those choices, task mix, and risk thresholds.

Measurement scope. The study measures tool-call counts, not LLM tokens, cost in dollars, human time, or network latency. Call count alone cannot establish actual financial efficiency.

Novelty and citations. The literature sample consists of identifiable foundational papers. Coverage of current 2025–2026 work remains incomplete; no claim of being first is justified.

Academic transparency. AI assistance was used to draft code and manuscript language. Authors should review code, rerun results, check references, and follow their institution and target venue disclosure policies.

9. Future Work and Release Plan

The immediate next stage is to validate the findings in at least two different tool domains with an open-weight instruction model running in Google Colab or a comparably documented environment. The LLM must generate tool calls and reason about observations rather than follow the scripted action sequence. The same evaluator should compare policies under identical tasks and faults. Tool APIs can be served by FastAPI; an MCP adapter can expose the same endpoints to compatible agents. Neither transport layer is an error-recovery method by itself.

Further tests should include unreliable verification endpoints, delayed replication, concurrent writes, longer task chains, learned or calibrated risk scores, explicit verification costs, and models with different tool-calling competence. A current literature audit and preregistered comparison plan should precede any strong originality or non-inferiority claim. An external reviewer should examine the code, output logs, and statistical choices.

10. Conclusion

In 1,600 deterministic simulation scenarios, a fixed-risk verification policy improved end-to-end success relative to no verification and prevented duplicate writes associated with post-commit timeouts. It also reduced tool-call overhead relative to unconditional verification, but missed plausible silent read corruptions and achieved substantially lower overall task success. The experiment therefore supports a nuanced conclusion: selective verification can help under certain modeled failures, but reliable detection of plausible data errors remains an open challenge. These results establish a reproducible pilot instrument, not general evidence about deployed LLM agents.

Data, Software, and Reproducibility Statement

The accompanying ZIP contains the simulator, fault definitions, unit tests, reproducible scenario logs, paired bootstrap code, summary metrics, graphics, and optional FastAPI/MCP adapters. Run `python run_experiment.py` from the package root; optional figures require Matplotlib. Results are deterministic for the documented Python seed and experimental version. The optional adapters were not used to produce this paper's measurements.

Publication and AI-use Notice

This is an unreviewed working paper documenting a reproducible scripted-policy simulation, not a peer-reviewed article or a validated result for LLM-powered agents. Public posting on ResearchGate or GitHub does not constitute scholarly peer review. The author should independently inspect the code and results, check current related work, and follow AI-assistance and preprint policies for any future journal or conference submission.

References

[1] Yao, S., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[2] Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761. https://arxiv.org/abs/2302.04761

[3] Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366. https://arxiv.org/abs/2303.11366

[4] Gou, Z., et al. (2023). CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. arXiv:2305.11738. https://arxiv.org/abs/2305.11738

[5] Liu, X., et al. (2023). AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688. https://arxiv.org/abs/2308.03688

[6] Yao, S., et al. (2024). tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045. https://arxiv.org/abs/2406.12045

[7] Zhou, S., et al. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854. https://arxiv.org/abs/2307.13854

[8] Jimenez, C. E., et al. (2023). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770. https://arxiv.org/abs/2310.06770

Appendix A. Proposed Real-LLM Validation Extension

STATUS: PLANNED AND CODE PREPARED; NO REAL LLM INFERENCE REPORTED IN THIS MANUSCRIPT.

This appendix describes a follow-up protocol intended to address the pilot's principal external-validity limitation. The scripted simulation results in Sections 1–8 remain unchanged. The LLM integration is not evidence of a completed LLM experiment.

A.1 Hypothesis and experimental setup

A language model will independently select read_stock, read_price, write_stock, or finish actions using a constrained JSON interface. The AAVF verification controller mediates tool responses. Tool failures are injected by the simulator; hidden labels are reserved for evaluation and are never passed to the model or the controller.

Four strategies will be evaluated on paired scenarios: no verification, bounded retry, selective adaptive verification, and verify-all. The LLM-selected action sequences and final task outcomes will be recorded. The initial model candidate is Qwen2.5-1.5B-Instruct, subject to Colab hardware and license availability.

A.2 Measurements and public benchmarks

Primary outcome: both final database state and model-reported values are correct. Secondary outcomes: duplicate writes, verification and tool calls, invalid tool actions, model call count, completed tasks, and runtime. Future work should add model token counts, multiple held-out task families, unreliable audit endpoints, and statistical uncertainty.

After validating the model-backed harness, a license-compatible stateful benchmark subset (for example, τ-bench) may be adapted for controlled fault injection. BFCL tests function-calling behavior but is not by itself a benchmark of recovery from ambiguous state-changing operations. Kaggle-hosted copies require source, license, and scoring checks.

A.3 Outstanding publication gates

- Complete an up-to-date 2025–2026 literature review; compare the closest verification and recovery methods before claiming novelty.
- Run the real-model inference experiment and preserve raw traces, exact model versions, task seeds, and scripts.
- Use official benchmark tasks and scoring where feasible; clearly distinguish synthetic from external benchmark outcomes.
- Compare strong baselines, report confidence intervals, and explain failures and risks of silent plausible tool errors.
- Obtain supervisor feedback, review references, and comply with the chosen publication venue's disclosure and licensing requirements.

Researcher: Roshaan Ahmed Khan | BS Artificial Intelligence (BSAI) | Sindh Madressatul Islam University, Pakistan